Deep Learning for Credit Risk Assessment

AI-100 Midterm Project

Prepared by:

Muhammad Adam Danial bin Mohd Nasir

March 1st 2026

Table of Contents

1. Introduction 4
2. Problem Definition 4
3. Dataset Design and Generation 5
3.1 Features 5
3.2 Label Generation Strategy 5
3.3 Dataset Size and Class Distribution 6
3.4 Dataset Creation Code 6
4. Deep Learning Model (MLP) 8
4.1 Network Architecture 9
4.2 Visual Representation 10
4.3 Forward Propagation 10
4.4 Loss Function 11
4.5 Backpropagation and Optimization 11
4.6 Why MLP is Appropriate for This Task 13
5. Training Setup (Implementation in train_mlp_v1.py) 13
5.1 Data Preparation and Splitting 13
5.2 Feature Scaling 15
5.3 Model Training Configuration 16
5.4 Evaluation Procedure and Visual Outputs 18
5.4.1 Training vs Validation Loss Curve 20
5.4.2 Confusion Matrix Heatmap 20
5.4.3 Predicted Probability Distribution Histogram 21
5.4.4 MLP Architecture diagram 21
6. Results and Evaluation 22
6.1 Confusion Matrix Report 22
6.2 Training vs Validation Loss Curve 23
6.3 Predicted Probability Distribution by Class 24
6.4 Classification Report Analysis 25
7. Problems and Fix 25
7.1 Identified Issue 25

7.2 Dataset refinement 26
7.3 Revised Model Results 27
7.3.1 Confusion Matrix Report 28
7.3.2 Training vs Validation Loss Curve 29
7.3.3 Predicted Probability Distribution by Class 30
7.3.4 Classification Report Analysis 31
8. Lessons Learned 32
8.1 The Importance of a Proper Dataset 32
8.2 Accuracy Can Be Misleading 32
8.3 Controlled Experimentation Matters 33
8.4 Probability Outputs Are More Informative Than Binary Labels 33
8.5 Nonlinearity Helps, But It Cannot Fix Bias 33
9. Conclusion 34
10. References 34

1. Introduction

Every day, banks and financial institutions make decisions about whether to approve or reject loan applications. A wrong decision can have serious consequences. Approving a high-risk applicant may lead to financial losses if the borrower defaults, while rejecting a low-risk applicant means turning away a reliable customer. Because of this, accurately assessing credit risk is a critical problem in modern finance.

Credit risk prediction can be formulated as a binary classification task. Given information about an applicant such as income, credit score, debt-to-income ratio, employment history, and past late payments, the goal is to predict whether the applicant is low risk or high risk. Traditionally, statistical models like logistic regression have been used for this purpose. However, real-world financial behavior is rarely perfectly linear. For example, a high loan amount may not be risky for someone with strong income and excellent credit, but the same loan could be extremely risky for someone with unstable employment and multiple late payments.

Deep learning models offer a way to capture these more complex relationships. By using multiple layers and nonlinear activation functions, neural networks can learn interactions between features instead of relying on a simple linear boundary. In this project, using ChatGPT as the main LLM, I generate a synthetic credit risk dataset and train a Multi-Layer Perceptron, or MLP, to classify loan applicants as high or low risk. The model's performance is evaluated using standard classification metrics, and its behavior is compared with a logistic regression baseline to better understand the benefits and trade-offs of deep learning.

2. Problem Definition

The goal of this project is to develop a binary classification model that predicts whether a loan applicant should be classified as high risk or low risk. In this context, "high risk" refers to applicants who are more likely to default on a loan, while "low risk" refers to applicants who are more likely to repay their loan on time.

Each applicant is represented by a set of numerical features that reflect their financial condition and borrowing profile. These features include annual income, credit score, debt-to-income ratio, number of late payments, employment history, loan amount, and age. Together, these variables provide a simplified but realistic representation of the factors typically considered in credit risk assessment.

Formally, let $X \in R$ represent the feature vector describing an applicant. The objective is to learn a function $f(X)$ that outputs a probability between 0 and 1, where values closer to 1 indicate higher risk. The final classification is obtained by applying a threshold to this probability. If the predicted probability exceeds the threshold, the applicant is labeled as high risk; otherwise, the applicant is labeled as low risk.

This is a supervised learning problem because each data point in the dataset is associated with a known label. The task of the model is to learn the relationship between the input features and the risk label based on training data, and then generalize this relationship to new, unseen applicants.

The primary objective is not only to achieve high accuracy, but also to evaluate how well the model identifies high-risk applicants. In real-world lending scenarios, failing to detect a high-risk borrower may lead to significant financial loss, making recall and precision important evaluation metrics in addition to overall accuracy.

3. Dataset Design and Generation

Since real-world credit data is often confidential and restricted, a synthetic dataset was generated to simulate a realistic credit risk scenario. The goal was to create applicant profiles that resemble real financial behavior while maintaining full control over the data generation process.

3.1 Features

Each applicant is described using the following numerical features:

- **Annual Income (USD):** 20,000 to 200,000
- **Credit Score:** 300 to 850
- **Debt-to-Income Ratio (DTI):** 0.0 to 1.0
- **Employment Years**: 0 to 30
- **Number of Late Payments**: 0 to 10
- **Loan Amount (USD):** 1,000 to 50,000
- **Age:** 18 to 70

These variables were selected because they are commonly used indicators in credit evaluation. For example, lower credit scores and higher debt-to-income ratios typically indicate greater financial risk.

3.2 Label Generation Strategy

Instead of assigning risk labels using a simple rule, a probabilistic scoring function was designed to introduce realistic variation. A risk score was computed based on weighted contributions from key features:

- Higher risk for:
    - Lower credit score
    - Higher debt-to-income ratio
    - More late payments
    - Larger loan amount relative to income

- Shorter employment history

The risk score was then passed through a sigmoid function to produce a probability between 0 and 1. Finally, labels were assigned by sampling from this probability distribution. This approach introduces controlled noise into the dataset, making the classification problem non-trivial and more reflective of real-world uncertainty.

3.3 Dataset Size and Class Distribution

The dataset consists of 5,000 synthetic loan applicants. The size was chosen to provide enough data for training a neural network while keeping the project computationally manageable. With several thousand samples, the model is able to learn meaningful patterns while maintaining stable training behavior.

This dataset represents the **initial** version used in the experimentation phase of the project. The intended class distribution was approximately **70 percent low risk** and **30 percent high risk**, reflecting a realistic lending scenario in which high-risk applicants are less common. This moderate imbalance was designed to mirror real-world credit data while still providing sufficient examples of high-risk cases for the model to learn meaningful patterns.

By generating the dataset in this way, the objective was to require the model to learn nonlinear relationships between financial indicators rather than relying on a simple threshold rule. This setup provides an appropriate foundation for evaluating the effectiveness of a Multi-Layer Perceptron in a binary classification task.

3.4 Dataset Creation Code

The synthetic dataset was generated programmatically using Python with the *NumPy* and *pandas* libraries. A fixed random seed was specified to ensure reproducibility, allowing the dataset to be regenerated consistently for repeated experimentation.

```
income = np.random.normal(loc=75000, scale=25000, size=N)
income = np.clip(income, a_min: 20000, a_max: 200000)

credit_score = np.random.normal(loc=680, scale=70, size=N)
credit_score = np.clip(credit_score, a_min: 300, a_max: 850)

debt_to_income = np.random.uniform( low: 0.0, high: 1.0, N)

employment_years = np.random.normal(loc=8, scale=5, size=N)
employment_years = np.clip(employment_years, a_min: 0, a_max: 30)

late_payments = np.random.poisson(lam=1.5, size=N)
late_payments = np.clip(late_payments, a_min: 0, a_max: 10)

loan_amount = np.random.normal(loc=15000, scale=8000, size=N)
loan_amount = np.clip(loan_amount, a_min: 1000, a_max: 50000)

age = np.random.normal(loc=40, scale=12, size=N)
age = np.clip(age, a_min: 18, a_max: 70)
```

Figure 1: *Feature generation*

In Figure 1, each feature was created using probability distributions chosen to reflect realistic financial behavior. Income, credit score, employment years, loan amount, and age were sampled from normal distributions and then clipped to remain within plausible ranges. Debt-to-income ratio was sampled from a uniform distribution between 0 and 1. The number of late payments was generated using a Poisson distribution to simulate count-based financial events. Clipping was applied to prevent unrealistic values such as negative income or out-of-range credit scores.

```
# ----------------------------
# Risk Score Calculation
# ----------------------------

# Normalize some variables for stability
income_norm = income / 200000
credit_norm = (850 - credit_score) / 550
loan_income_ratio = loan_amount / (income + 1)

# Risk formula (nonlinear interactions included)
risk_score = (
    2.5 * credit_norm +
    3.0 * debt_to_income +
    1.5 * late_payments / 10 +
    2.0 * loan_income_ratio -
    1.2 * employment_years / 30 +
    0.5 * (debt_to_income * credit_norm)  # nonlinear interaction
)
```

Figure 2: *Risk score calculation*

In Figure 2, the risk scoring function was implemented directly in code as a weighted mathematical expression combining normalized financial indicators. Each coefficient in the formula represents the relative importance of that feature in contributing to overall credit risk.

Firstly, the term **2.5 × credit_norm** assigns substantial weight to credit score because it is one of the strongest predictors of default risk in real-world lending. Lower credit scores are strongly associated with higher default probability, so this variable is given a relatively high coefficient to reflect its importance. Secondly, the coefficient **3.0 × debt_to_income** is slightly larger, indicating that debt burden relative to income is a critical risk factor. Applicants with high debt-to-income ratios are more financially constrained and therefore more vulnerable to repayment difficulties. Assigning a strong weight ensures this feature meaningfully influences the risk outcome.

Thirdly, the term **1.5 × late_payments / 10** captures past repayment behavior. While payment history is important, it is given a moderate weight compared to credit score and debt ratio, since late payments are already partially reflected in credit scores. Fourthly, the coefficient **2.0 × loan_income_ratio** reflects the financial strain of borrowing relative to earnings. Larger loan amounts relative to income increase repayment risk, so this variable is assigned a substantial but slightly lower weight than debt-to-income.

Fifthly, the term **1.2 × employment_years / 30** captures employment stability. Longer employment history generally indicates financial stability; however, it is assigned a smaller coefficient because it is a secondary predictor compared to credit score and debt levels. Finally, the nonlinear interaction term **0.5 × (debt_to_income × credit_norm)** introduces feature interaction. This means that the impact of debt burden depends partly on credit quality. For example, a high debt-to-income ratio may be riskier when combined with a weak credit profile. The interaction coefficient is smaller (**0.5**) to ensure it influences the score without overpowering the primary linear effects.

The coefficients were selected heuristically to simulate realistic financial behavior rather than being derived from empirical estimation. Their relative magnitudes control how strongly each financial factor contributes to the final risk probability.

```python
# Convert to probability using sigmoid
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

risk_probability = sigmoid(risk_score)

# ----------------------------
# Generate Labels
# ----------------------------

labels = np.random.binomial(1, risk_probability)

# ----------------------------
# Create DataFrame
# ----------------------------

df = pd.DataFrame({
    "income": income,
    "credit_score": credit_score,
    "debt_to_income": debt_to_income,
    "employment_years": employment_years,
    "late_payments": late_payments,
    "loan_amount": loan_amount,
    "age": age,
    "high_risk": labels
})
```

Figure 3: *Sigmoid function for computing risk score*

Moving on to Figure 3, the computed risk score was transformed into a probability using the sigmoid function. Final binary labels were generated by sampling from a binomial distribution using this probability. This probabilistic labeling approach introduces controlled variability into the dataset, preventing perfect separability and better reflecting real-world uncertainty in credit evaluation. The completed dataset was then exported as a CSV file for use in model training and evaluation.

4. Deep Learning Model (MLP)

To model the credit risk classification problem, a Multi-Layer Perceptron (MLP) was implemented using *TensorFlow* and *Keras* in Python. An MLP is a feedforward neural network composed of fully connected (dense) layers that transform input data through weighted linear combinations followed by nonlinear activation functions (GeeksForGeeks, 2025). Unlike logistic regression, which produces a strictly linear decision boundary, an MLP can learn complex nonlinear relationships between features.

This capability is particularly important in credit risk prediction. Financial indicators do not operate independently. For example, a high debt-to-income ratio may be significantly riskier when combined with a low credit score and a large loan amount, but less risky when income and employment stability are strong. A neural network can automatically learn these interaction effects without manually defining feature combinations.

4.1 Network Architecture

The architecture used in this project consists of:

1. **Input Layer**

The input layer corresponds to the seven numerical financial features which are income, credit score, debt-to-income ratio, employment years, late payments, loan amount, and age.

2. **Hidden Layer 1 (Dense, 64 Units, ReLU)**

This layer has 64 neurons with ReLU (Rectified Linear Unit) activation. ReLU introduces nonlinearity by computing:

$$f(z) = \max(0, z)$$

Figure 4: *Rectified Linear Unit*

This fully connected layer transforms the seven inputs into 64 learned representations.

3. **Dropout Layer (0.30)**

A dropout rate of 30% was applied to randomly deactivated neurons during training. This reduces overfitting by preventing the network from relying too heavily on specific neurons.

4. **Hidden Layer 2 (Dense, 32 units, ReLU)**

This layer has 32 neurons with ReLU activation. This further refines feature representations into 32 nonlinear combinations.

5. **Dropout Layer (0.20)**

A second dropout layer further improves generalization.

6. **Output Layer**

A single neuron with sigmoid activation, producing an output probability between 0 and 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Figure 5: *Sigmoid function*

This value represents the predicted probability that an applicant is classified as high risk.

4.2 Visual Representation

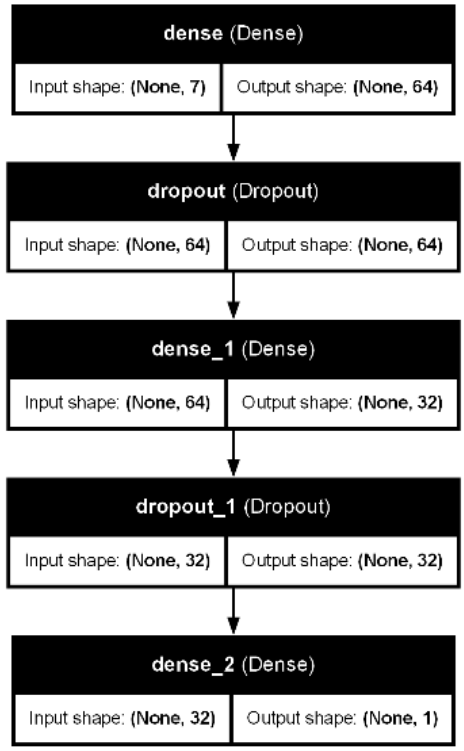

Figure 6: *MLP Architecture of Version 1*

4.3 Forward Propagation

During forward propagation, the input features are passed through each layer.

For each neuron, a weighted sum is computed:

$$z = \sum_i w_i x_i + b$$

Figure 7: *Linear Combination of Inputs (Neuron Pre-Activation Equation)*

where:

- x_i are input features,
- w_i are learnable weights,
- b is a bias term.

The result is then passed through an activation function (ReLU in hidden layers, sigmoid in the output layer). The final output is a probability estimate for the high-risk class.

4.4 Loss Function

Because the task is binary classification, **binary cross-entropy** was used as the loss function:

$$L = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right]$$

Figure 8: *Binary Cross-Entropy Loss Function*

where:

- y_i is the true label
- $\hat{y}_i$ is the predicted probability
- N is the number of samples

Binary cross-entropy penalizes confident but incorrect predictions more heavily than small errors, making it suitable for probabilistic classification tasks.

4.5 Backpropagation and Optimization

The MLP is trained using backpropagation, a supervised learning algorithm that minimizes the binary cross-entropy loss by computing gradients of the loss function with respect to each weight and bias in the network. After forward propagation produces predicted probabilities, the loss is calculated, and the error is propagated backward through the layers using the chain rule of calculus.

For a given weight w, the basic gradient descent update rule is:

$$w_{t+1} = w_t - \alpha \frac{\partial L}{\partial w_t}$$

Figure 9: *Gradient descent weight update rule*

where:

- α is the learning rate,
- $\frac{\partial L}{\partial w_t}$ is the gradient of the loss with respect to the weight at time step t.

Instead of using standard gradient descent, this project employs the Adaptive Moment Estimation (Adam) optimizer, which improves convergence speed and stability by combining momentum and adaptive learning rates (GeeksForGeeks, 2025).

Adam maintains two exponentially weighted moving averages:

1. **First moment (mean of gradients):**

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \frac{\partial L}{\partial w_t}$$

Figure 10: *First moment (momentum) update*

This term acts similarly to momentum, smoothing gradient updates and reducing oscillations during training.

2. **Second moment (mean of squared gradients):**

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \left(\frac{\partial L}{\partial w_t} \right)^2$$

Figure 11: *Second moment (variance) update*

This allows the optimizer to scale learning rates individually for each parameter. Parameters with large gradients receive smaller effective updates, while parameters with smaller gradients receive relatively larger updates.

To correct for initialization bias in early iterations, bias-corrected estimates are computed:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Figure 12: *Bias correction for first and second moments*

The final update rule becomes:

$$w_{t+1} = w_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Figure 13: *Final Adam optimizer parameter update equation*

where ϵ is a small constant added to prevent division by zero.

Adam was selected because it automatically adjusts the learning rate for each parameter during training. Instead of using a single fixed step size for all weights, Adam adapts how much each weight should change based on past gradient information. This is helpful because different features may influence the model at different scales. For example, income and credit score operate on different numerical ranges, and their associated gradients can vary significantly.

In addition, Adam reduces instability during the early stages of training. When a neural network first begins learning, gradients can fluctuate or oscillate. By using momentum and adaptive scaling, Adam smooths these updates, preventing extremely large or erratic weight changes.

Adam basically helps the model learn in a more controlled and efficient way. It moves quickly when the direction is consistent and slows down when gradients are unstable. This allows the network to converge faster and more reliably, which is especially useful for nonlinear models like the MLP used in this credit risk classification task.

4.6 Why MLP is Appropriate for This Task

The credit risk problem is inherently **nonlinear** because financial risk does not increase in a simple additive way. For example, a high debt-to-income ratio may not always indicate high risk on its own. However, when combined with a low credit score and multiple late payments, the overall risk can increase significantly. In other words, the effect of one variable often depends on the values of other variables. These interaction effects cannot be fully captured by a simple linear model.

By stacking multiple dense layers with nonlinear activation functions, the MLP is able to learn these interactions automatically. Each hidden layer transforms the input data into more abstract representations, allowing the model to detect patterns that are not immediately visible in the raw features.

The MLP also produces probabilistic outputs through the sigmoid activation function, meaning it does not simply classify applicants as high or low risk. Instead, it estimates the likelihood that an applicant belongs to the high-risk category. This probabilistic interpretation is important in financial settings, where decisions often depend on risk thresholds rather than absolute labels.

Additionally, neural networks can adapt to moderate class imbalance by learning decision boundaries based on patterns in the data rather than relying on strict linear separation. While they are not immune to imbalance issues, they provide flexibility in modeling overlapping classes.

Overall, these characteristics make the MLP a suitable deep learning approach for structured tabular credit risk classification, particularly when feature interactions and nonlinear relationships are present.

5. Training Setup (Implementation in train_mlp_v1.py)

5.1 Data Preparation and Splitting

The training script begins by importing the required libraries (Figure 14). The *os* module is used for file handling and directory management. The *pandas* library is imported to load and manipulate

the dataset. The *matplotlib.pyplot* library is used to generate visual outputs such as the loss curve, confusion matrix heatmap, and probability distribution histogram.

```
import os
import pandas as pd
import matplotlib.pyplot as plt

# Traditional ML utilities
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score

# Deep Learning (TensorFlow / Keras)
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.utils import plot_model
```

Figure 14: *Import statements*

From *scikit-learn*, **train_test_split** is imported to divide the dataset into training and testing subsets. **StandardScaler** is used for feature standardization, while **accuracy_score**, **confusion_matrix**, and **classification_report** are imported for model evaluation.

For deep learning implementation, *TensorFlow* and *Keras* are imported. The **layers** module provides Dense and Dropout layers, and **plot_model** is used to generate a visual diagram of the network architecture. After importing the libraries, the script ensures that a folder named **visuals** exists (Figure 15).

```
os.makedirs( name: "visuals", exist_ok=True)
```

Figure 15: *Make directory logic*

The dataset **synthetic_credit_risk_v1.csv** is then loaded using **pd.read_csv()** (Figure 16). A file existence check is performed before loading to prevent runtime errors.

```
# Rename the path to match the dataset file
data_path = "data/synthetic_credit_risk_v1.csv"

# Check if dataset exists
if not os.path.exists(data_path):
    raise FileNotFoundError(
        "Dataset not found. Please run dataset_creation_v1.py first."
    )

df = pd.read_csv(data_path)
```

Figure 16: *CSV read file using pandas*

In Figure 17, the target column **high_risk** is separated from the input features. All other columns are assigned to X, and the binary labels are assigned to y. The dataset is then split into training and

testing sets using a **75/25 split**, meaning that 75% of the data is used to train the model while the remaining 25% is reserved for evaluating its performance on unseen data. This approach ensures that the model is assessed on data it has not encountered during training, providing a more realistic estimate of generalization performance. Given that the dataset contains 5,000 samples, this results in 3,750 training examples and 1,250 test examples, which offers a strong balance between learning capacity and evaluation reliability.

```
# ---------------------------------------------------------
# 2. Define Features (X) and Target (y)
# ---------------------------------------------------------
target_col = "high_risk"

X = df.drop(columns=[target_col])  # All input features
y = df[target_col].astype(int)     # Binary labels (0 or 1)

# ---------------------------------------------------------
# 3. Train/Test Split
# ---------------------------------------------------------
# Stratify ensures class ratio (70/30) is preserved
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y,
    test_size=0.25,
    random_state=42,
    stratify=y
)
```

Figure 17: *Define features and targets*

Stratified sampling (**stratify=y**) ensures that the class distribution remains consistent in both subsets (Figure 17). In other words, the proportion of low-risk and high-risk applicants in each subset mirrors the overall dataset. This is important in classification problems, especially when classes are not perfectly balanced, because it prevents one subset from containing disproportionately more samples of a particular class. Maintaining similar class ratios allows the model to be trained and evaluated under realistic and consistent conditions.

Additionally, a fixed random state (**random_state=42**) guarantees reproducibility. This guarantees that the dataset is split in the same way each time the script is executed. Without a fixed seed, different runs could produce slightly different splits and therefore different performance outcomes, making experimental comparisons less reliable.

5.2 Feature Scaling

Once the dataset is split, feature scaling is performed using **StandardScaler** (Figure 18). The scaler is first fitted on the training data using **scaler.fit_transform(X_train)** and then applied to both training and testing sets using **scaler.transform(X_test)**.

```python
# ---------------------------------------------------------
# 4. Feature Scaling
# ---------------------------------------------------------
# Neural networks are sensitive to feature scale.
# StandardScaler transforms features to mean=0, std=1.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Figure 18: *Feature scaling*

Standardization converts each feature to have a mean of 0 and a standard deviation of 1. This means that all features are placed on a similar numerical scale before being fed into the neural network. This step is important because neural networks are sensitive to the size of input values. If some features have very large numbers, such as income or loan amount, while others have small numbers, those larger features can dominate the learning process. This can cause unstable training or slow convergence. By scaling the features to a common range, the model can learn from all variables more evenly and efficiently.

5.3 Model Training Configuration

The model architecture is then defined using *Keras* **Sequential** (Figure 19). A fixed random seed (**tf.random.set_seed(42))** is set to reduce variability in weight initialization. Neural networks begin training with randomly assigned weights, and different initial values can influence how the model learns and converges. This complements the earlier use of **random_state=42** during the data split, as both settings control different sources of randomness within the experiment. In other words, one fixes how the data is divided, while the other fixes how the model begins learning.

```python
# ---------------------------------------------------------
# 5. Build the Multi-Layer Perceptron (MLP)
# ---------------------------------------------------------
# Architecture:
# Input Layer -> 64 neurons -> 32 neurons -> 1 output neuron
# ReLU activation introduces non-linearity
# Sigmoid outputs probability between 0 and 1

tf.random.set_seed(42)

model = keras.Sequential([
    layers.Input(shape=(X_train_scaled.shape[1],)),  # input dimension
    layers.Dense( units: 64, activation="relu"),
    layers.Dropout(0.30),  # reduces overfitting
    layers.Dense( units: 32, activation="relu"),
    layers.Dropout(0.20),
    layers.Dense( units: 1, activation="sigmoid")  # binary classification output
])
```

Figure 19: *Building the MLP*

The network consists of a fully connected Dense layer with 64 neurons using ReLU activation, followed by a Dropout layer with a 30% dropout rate to reduce overfitting. A second Dense layer

with 32 neurons and ReLU activation is then applied to further refine the learned feature representations, followed by another Dropout layer with a 20% dropout rate to improve generalization. Finally, the network concludes with a Dense output layer containing a single neuron with sigmoid activation, which produces a probability between 0 and 1 representing the likelihood that an applicant is classified as high risk.

```
# ---------------------------------------------------------
# 6. Compile the Model
# ---------------------------------------------------------
# Binary Cross Entropy is used for binary classification
# Adam optimizer performs adaptive gradient updates

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

Figure 20: *Compiling the model*

The model is then compiled using the Adam optimizer with a learning rate of 0.001 and binary cross-entropy as the loss function (Figure 19). The learning rate controls the magnitude of each weight update during optimization, and a value of 0.001 provides a balance between stable convergence and efficient training. Accuracy is also included as a monitoring metric to track classification performance throughout training.

When the program is executed, the model is trained using **50 epochs**, a **batch size of 32**, a 20% validation split (**validation_split=0.2**), and a parameter **verbose=1**.

```
# ---------------------------------------------------------
# 7. Train the Model
# ---------------------------------------------------------
# validation_split=0.2 reserves 20% of training data for validation
# Epochs determine how many times the model sees the dataset

history = model.fit(
    X_train_scaled,
    y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=32,
    verbose=1
)
```

Figure 21: *Training the model*

An epoch represents one complete pass through the entire training dataset (Figure 21). In this case, 50 epochs mean the model processes the full training data 50 times, gradually adjusting its weights

after each pass to reduce the loss. This means that the model studies the same data multiple times to improve its understanding.

Next, we have the **batch size of 32** which determines how many samples are processed before the model updates its weights. Instead of updating weights after every single sample, the model processes 32 samples at a time, computes the average gradient, and then updates the weights. In simple words, the model learns in small groups rather than one example at a time, which makes training more stable and computationally efficient.

The **validation split of 20%** means that 20% of the training data is temporarily set aside and used to evaluate the model during training. This data is not used to update weights; instead, it is used to measure how well the model generalizes to unseen data after each epoch. This allows early detection of overfitting, which occurs when the model performs well on training data but poorly on new data. In simple terms, the validation set acts like a practice test during learning, helping check whether the model is truly improving or just memorizing the training data.

Lastly, the parameter **verbose=1** controls how training progress is displayed in the console. When set to 1, the model prints a progress bar and performance metrics (such as loss and accuracy) after each epoch in the Python terminal (Figure 22). This allows monitoring of training behavior in real time.

```
Epoch 1/50
94/94 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8707 - loss: 0.4150 - val_accuracy: 0.9147 - val_loss: 0.2763
Epoch 2/50
94/94 ━━━━━━━━━━━━━━━━━━━━ 0s 984us/step - accuracy: 0.9057 - loss: 0.3025 - val_accuracy: 0.9147 - val_loss: 0.2621
Epoch 3/50
94/94 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9057 - loss: 0.2906 - val_accuracy: 0.9147 - val_loss: 0.2605
Epoch 4/50
94/94 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9057 - loss: 0.2920 - val_accuracy: 0.9147 - val_loss: 0.2607
Epoch 5/50
94/94 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9057 - loss: 0.2803 - val_accuracy: 0.9147 - val_loss: 0.2619
Epoch 6/50
94/94 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9057 - loss: 0.2805 - val_accuracy: 0.9147 - val_loss: 0.2614
Epoch 7/50
94/94 ━━━━━━━━━━━━━━━━━━━━ 0s 973us/step - accuracy: 0.9057 - loss: 0.2812 - val_accuracy: 0.9147 - val_loss: 0.2619
```

Figure 22: *Epoch progress bars*

5.4 Evaluation Procedure and Visual Outputs

After training, the model generates predicted probabilities for the test set using **model.predict(X_test_scaled).ravel()** in Figure 23.

```python
# ----------------------------------------------------------
# 8. Evaluate on Test Data
# ----------------------------------------------------------
# Predict probabilities
y_prob = model.predict(X_test_scaled).ravel()

# Convert probabilities to class labels using threshold 0.5
y_pred = (y_prob >= 0.5).astype(int)

# Compute evaluation metrics
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred, digits=3)

print("\n===== Test Results =====")
print(f"Accuracy: {acc:.3f}")
print("\nConfusion Matrix:")
print(cm)
print("\nClassification Report:")
print(report)
```

Figure 23: *Test data evaluation*

These probabilities are converted into binary class predictions using a **threshold of 0.5** (Figure 23). If the predicted probability is greater than or equal to 0.5, the applicant is classified as high risk (1); otherwise, the applicant is classified as low risk (0). The threshold of 0.5 is a standard choice for binary classification when no specific cost-sensitive adjustment is applied. In simple terms, values above 50% are treated as "high risk," while values below 50% are treated as "low risk."

The model's performance is evaluated using **accuracy**, **confusion matrix**, and **classification report metrics** (Figure 23). Firstly, accuracy measures the proportion of total predictions that were correct. It provides an overall performance summary but may be misleading when class imbalance exists. Secondly, confusion matrix provides a detailed breakdown of predictions, including true positives, true negatives, false positives, and false negatives. This helps identify whether the model is better at predicting low-risk or high-risk applicants Thirdly, classification report includes precision, recall, and F1-score for each class.

In the classification report, **precision** measures how many predicted high-risk applicants were actually high risk. **Recall**, on the other hand, measures how many actual high-risk applicants were correctly identified. **F1-scores** balances precision and recall into a single metric. Together, these metrics provide a comprehensive evaluation of the model's predictive performance and allow deeper analysis beyond overall accuracy. The script then generates and saves four visual outputs.

5.4.1 Training vs Validation Loss Curve

```python
# ----------------------------------------------------------
# 9. Plot Training vs Validation Loss
# ----------------------------------------------------------
# This shows learning behavior and possible overfitting

plt.figure(figsize=(8, 5))
plt.plot( *args: history.history["loss"], label="Training Loss")
plt.plot( *args: history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Binary Cross-Entropy Loss")
plt.title("MLP Training and Validation Loss (v1)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig( *args: "visuals/mlp_loss_curve_v1.png", dpi=200)
plt.show()

print("\nLoss curve saved as visuals/mlp_loss_curve_v1.png")
```

Figure 24: *Code*

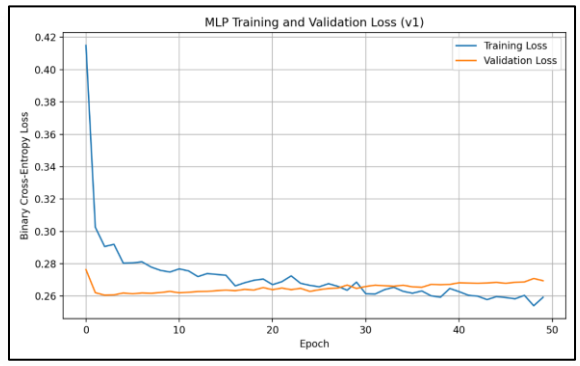

Figure 25: *MLP Training and Validation Loss Curve (Sample)*

The loss curve plots binary cross-entropy loss for both the training set and validation set across all epochs (Figure 25). This visualization shows how the model's error decreases as learning progresses. If both curves decrease and remain close to each other, the model is learning effectively and generalizing well. However, if the training loss continues decreasing while validation loss begins increasing, this indicates overfitting. So, this graph shows whether the model is improving properly or just memorizing the training data.

5.4.2 Confusion Matrix Heatmap

```python
# ----------------------------------------------------------
# 10. Confusion Matrix Heatmap (no seaborn required)
# ----------------------------------------------------------
plt.figure(figsize=(6, 5))
plt.imshow(cm, interpolation="nearest")
plt.title("Confusion Matrix (MLP v1)") # Change for every version
plt.colorbar()

# Axis tick labels
plt.xticks( ticks: [0, 1],  labels: ["Pred Low (0)", "Pred High (1)"])
plt.yticks( ticks: [0, 1],  labels: ["Actual Low (0)", "Actual High (1)"])

# Add counts inside the boxes
for i in range(2):
    for j in range(2):
        plt.text(j, i, cm[i, j], ha="center", va="center")

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.tight_layout()
plt.savefig( *args: "visuals/mlp_confusion_matrix_v1.png", dpi=200)
plt.show()

print("Saved: visuals/mlp_confusion_matrix_v1.png")
```

Figure 26: *Code*

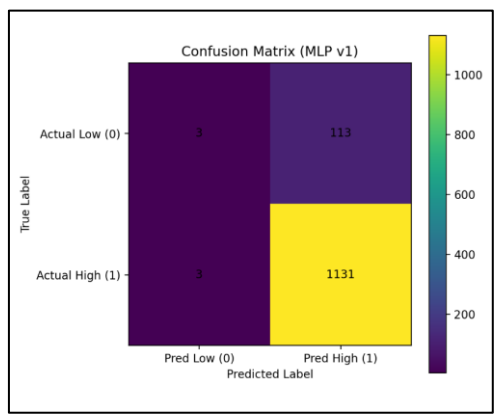

Figure 27: *Confusion Matrix (Sample)*

The confusion matrix heatmap provides a visual breakdown of prediction outcomes, including true positives, true negatives, false positives, and false negatives (Figure 27). The color intensity represents the number of samples in each category, making it easier to identify prediction strengths and weaknesses. This visualization helps determine whether the model is better at identifying low-risk or high-risk applicants and whether misclassifications are occurring disproportionately in one class.

5.4.3 Predicted Probability Distribution Histogram

```python
# ----------------------------------------------------------
# 11. Predicted Probability Distribution Histogram
# ----------------------------------------------------------
# Split predicted probabilities by true class
low_probs = y_prob[y_test.values == 0]
high_probs = y_prob[y_test.values == 1]

plt.figure(figsize=(8, 5))
plt.hist(low_probs, bins=30, alpha=0.7, label="Actual Low Risk (0)")
plt.hist(high_probs, bins=30, alpha=0.7, label="Actual High Risk (1)")
plt.xlabel("Predicted Probability of High Risk")
plt.ylabel("Count")
plt.title("Predicted Probability Distribution by Class (v1)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig( *args: "visuals/mlp_probability_distribution_v1.png", dpi=200)
plt.show()

print("Saved: visuals/mlp_probability_distribution_v1.png")
```

Figure 26: *Code*

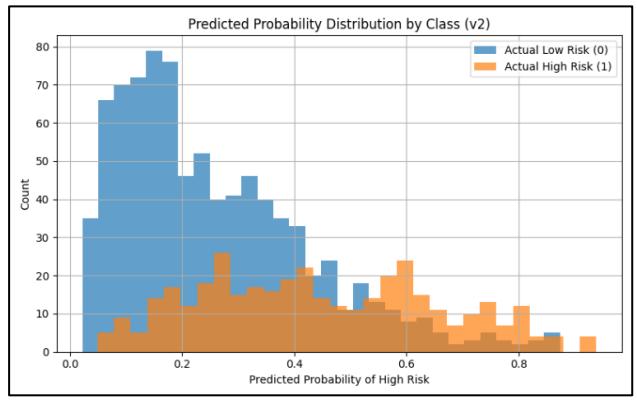

Figure 27: *Predicted Probability Distribution Histogram (Sample)*

The probability distribution histogram displays the predicted probabilities for low-risk and high-risk applicants separately (Figure 27). Ideally, the two distributions should be well separated, indicating that the model assigns high probabilities to high-risk cases and low probabilities to low-risk cases. Overlap between distributions suggests classification difficulty. So, this graph shows how confidently the model distinguishes between the two classes.

5.4.4 MLP Architecture diagram

```python
# ----------------------------------------------------------
# 12. MLP Architecture
# ----------------------------------------------------------

plot_model(
    model,
    to_file="visuals/mlp_architecture_v1.png",
    show_shapes=True,
    show_layer_names=True,
    dpi=200
)

print("Saved: visuals/mlp_architecture_v1.png")
```

Figure 28: *Code for architecture*

The architecture diagram provides a structural visualization of the neural network, including layer types, number of neurons, and tensor shapes. This ensures transparency in model design and allows readers to verify the architecture described in the methodology section. It serves as a structural summary of the model implementation. The code in Figure 28 help generated the diagram in Figure 6.

6. Results and Evaluation

6.1 Confusion Matrix Report

```
===== Test Results =====
Accuracy: 0.906

Confusion Matrix:
[[   1  115]
 [   3 1131]]

Classification Report:
              precision    recall  f1-score   support

           0      0.250     0.009     0.017       116
           1      0.908     0.997     0.950      1134

    accuracy                          0.906      1250
   macro avg      0.579     0.503     0.484      1250
weighted avg      0.847     0.906     0.864      1250
```

Figure 29: *Terminal results for V1*

When **train_mlp_v1.py** is executed, the program prints a summary of test performance in the terminal (Figure 29) and then generates the training and evaluation plots. The terminal output reports an overall accuracy of **0.906** which is high, followed by a confusion matrix and a classification report. Although these metrics are part of evaluation, they are still useful to mention here because they confirm that the model successfully trained and produced stable predictions rather than failing to learn.

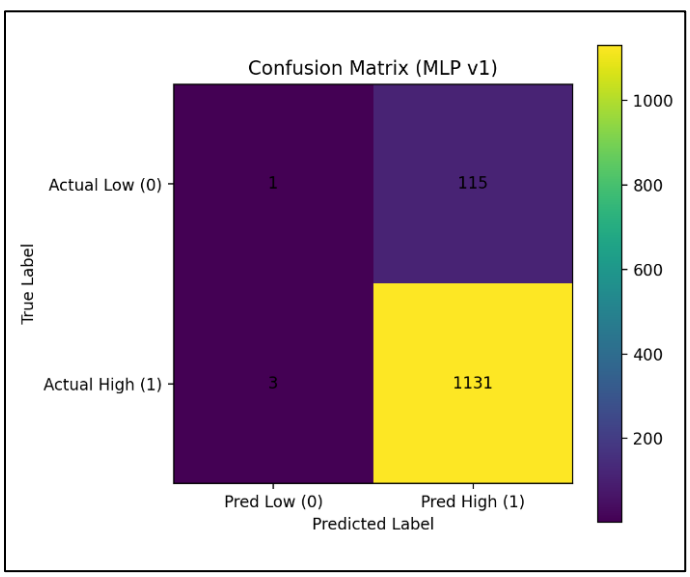

Figure 30: *Confusion Matrix for V1*

From the confusion matrix, each value has a specific meaning:

- **1 (top-left)**
  - True Negatives (TN): actual low-risk (0) correctly predicted as low-risk (0)
- **115 (top-right)**
  - False Positives (FP): actual low-risk (0) incorrectly predicted as high-risk (1)
- **3 (bottom-left)**
  - False Negatives (FN): actual high-risk (1) incorrectly predicted as low-risk (0)
- **1131 (bottom-right)**
  - True Positives (TP): actual high-risk (1) correctly predicted as high-risk (1)

This output shows that the model predicted **almost everything as high risk**, which explains why accuracy appears high. The model correctly identifies nearly all high-risk cases (**large TP = 1131**), but it almost completely fails to identify low-risk cases (**only 1 correct low-risk prediction out of 116 low-risk examples in the test set**). Even before looking at curves, this already suggests that the model's learned decision behavior is highly skewed.

6.2 Training vs Validation Loss Curve

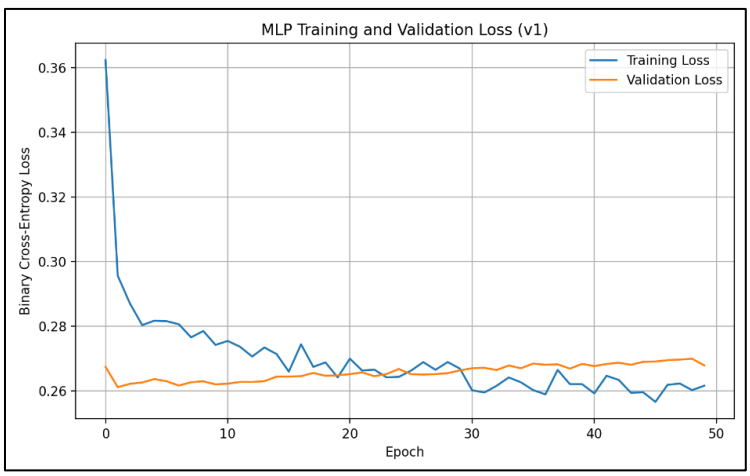

Figure 31: *MLP Training and Validation Loss (V1)*

The training behavior is visualized in the training vs validation loss curve (Figure 31). Both curves represent binary cross-entropy loss, which measures how far the predicted probabilities are from the correct labels. Lower loss means better fitting. The training loss begins at about **0.36** and drops sharply during the first few epochs to around **0.29**. This steep initial decline is expected because the model starts from random weights and quickly learns the strongest signals in the data. After that, the curve becomes flatter and gradually decreases toward roughly **0.26**, indicating that the model is approaching convergence where learning continues, but improvements become smaller.

The validation loss begins around **0.267**, decreases slightly early on, and then remains mostly flat around the **0.26 to 0.27** range. Near later epochs, it trends slightly upward (**ending around ~0.268 to 0.27),** while training loss continues to decrease slowly.

The overall pattern suggests mild **overfitting**. Training loss continues improving while validation loss plateaus and slightly increases. However, the gap is not extremely large, and the validation loss does not rise sharply, so this is not severe overfitting. Instead, the more important issue is that the model's learning is converging toward a decision strategy that favors predicting the majority outcome (high risk) in this dataset version.

6.3 Predicted Probability Distribution by Class

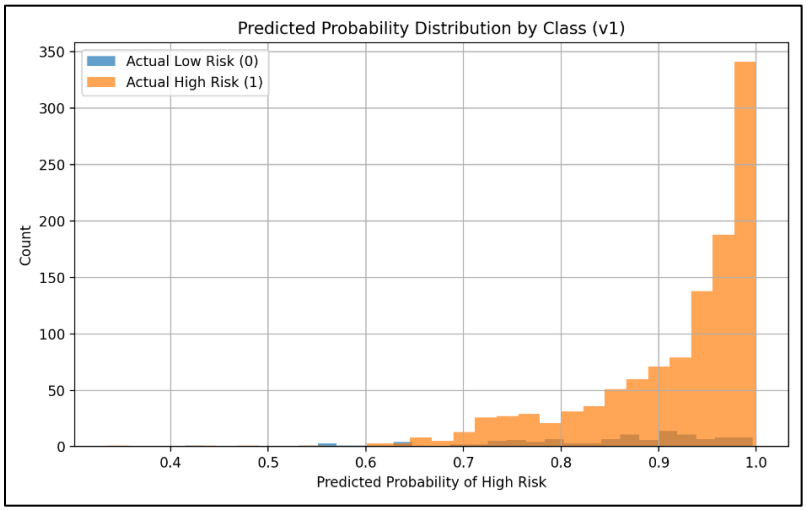

Figure 32: *Predicted Probability Distribution by Class (V1)*

As for the predicted probability distribution histogram, in Version 1, the histogram reveals a strong skew toward high probability values. Most predictions for actual high-risk applicants are concentrated very close to **1.0**, indicating extremely confident high-risk classifications. However, predictions for actual low-risk applicants are also largely concentrated at high probability values, with many exceeding **0.8** or even **0.9**. Very few predictions fall below the **0.5** threshold.

The shape of the distribution shows substantial overlap between the two classes and a heavy rightward shift toward probability values near **1.0**. In a well-performing binary classifier, low-risk applicants would cluster near 0 and high-risk applicants near 1, with clear separation between the two distributions. Instead, both classes are grouped toward the high-probability region, indicating that the model has learned a **biased decision pattern**. When the 0.5 threshold is applied, nearly all samples are classified as high risk, which directly explains the confusion matrix results.

This visualization confirms that the high overall accuracy is misleading. Although the model appears to perform well numerically, the probability distribution demonstrates that it lacks meaningful class discrimination. The model is highly confident but systematically biased toward

predicting the majority class, resulting in near-perfect recall for high-risk applicants and almost complete failure to correctly identify low-risk applicants.

6.4 Classification Report Analysis

The classification report printed in the terminal (Figure 29) provides deeper insight into the model's behavior beyond overall accuracy. Although the model achieved an overall accuracy of **0.906**, the class-wise metrics reveal severe imbalance in prediction behavior.

For Class 0 (Low Risk):

- Precision: 0.250
- Recall: 0.009
- F1-score: 0.017
- Support: 116

The recall of **0.009** indicates that the model correctly identifies **less than 1%** of actual low-risk applicants. Out of **116** true low-risk cases in the test set, only **1** was correctly classified. The precision of **0.250** means that when the model predicts low risk, it is correct only **25%** of the time. However, since the model almost never predicts low risk, this metric is based on extremely few predictions. The F1-score of **0.017** is extremely low, confirming that the model essentially fails to recognize low-risk applicants. This demonstrates that the model has almost completely collapsed on Class 0.

For Class 1 (High Risk):

- Precision: 0.908
- Recall: 0.997
- F1-score: 0.950
- Support: 1134

For high-risk applicants, performance appears nearly perfect. The recall of **0.997** means that almost every actual high-risk case is correctly identified. Out of **1134** high-risk examples, only **3** were misclassified. The precision of **0.908** indicates that when the model predicts high risk, it is correct over **90%** of the time. The F1-score of **0.950** is extremely high, suggesting strong performance for this class. However, this apparent strength is misleading because the dataset itself is dominated by high-risk labels.

7. Problems and Fix

7.1 Identified Issue

The primary issue identified in Version 1 of the model was severe class imbalance in the generated dataset. Although the dataset was intended to reflect a realistic lending scenario, the test set

contained **1,134** high-risk applicants and only **116** low-risk applicants. This extreme imbalance caused the model to learn a biased decision boundary that heavily favored predicting the majority class (high risk). As shown in the confusion matrix (Figure 30), the model correctly identified nearly all high-risk cases but failed almost entirely to recognize low-risk applicants, producing only **1 true negative out of 116** actual low-risk cases.

This imbalance resulted in a misleadingly high overall accuracy of **0.906** (Figure 29). While accuracy measures the proportion of correct predictions, it does not account for how predictions are distributed across classes. In imbalanced datasets, a model can achieve high accuracy simply by predicting the dominant class for most samples. In this case, the model predicted almost all applicants as high risk, which was sufficient to produce high accuracy due to the overwhelming presence of high-risk samples in the dataset.

However, deeper evaluation metrics revealed the underlying problem. The recall for **class 1 (high risk)** was **0.997**, indicating nearly perfect identification of high-risk applicants (Figure 29). In contrast, the recall for **class 0 (low risk)** was only **0.009**, meaning the model failed to detect almost all low-risk cases. This demonstrates that accuracy alone was not an appropriate performance indicator for this classification task.

So, the issue was not a failure of the neural network architecture itself, but rather a flaw in the dataset distribution. The imbalance dominated the model's learning behavior, leading it to optimize for the majority class at the expense of meaningful class discrimination.

7.2 Dataset refinement

To correct the imbalance observed in Version 1, the dataset generation process was refined while keeping the overall experimental setup consistent. The same random seed (**np.random.seed(42)**) and dataset size (**N = 5000**) were retained so that any performance improvements could be attributed specifically to the change in label generation rather than random variation. If the seed had been changed, the dataset would have been generated differently, and it would be unclear whether improvements were caused by better calibration or simply by chance. Keeping these factors constant ensures a fair comparison between Version 1 and Version 2.

```python
def calibrate_bias(risk_score, target_rate=0.30, max_iter=60):  1 usage  admdan
    """
    Finds a bias term 'b' such that mean(sigmoid(risk_score + b)) ~= target_rate.
    Uses binary search because sigmoid is monotonic with respect to b.
    """
    lo, hi = -10.0, 10.0  # wide enough range for shifting probabilities

    for _ in range(max_iter):
        mid = (lo + hi) / 2
        probs = sigmoid(risk_score + mid)
        rate = probs.mean()

        if rate > target_rate:
            hi = mid
        else:
            lo = mid

    return (lo + hi) / 2
```

Figure 33: *Bias calibration function for V2*

The key refinement in Version 2 is that the original risk score formula was preserved, but the probability mapping was calibrated to achieve a target high-risk rate of approximately **30%** (Figure 33). In Version 1, the sigmoid transformation of the raw risk score unintentionally produced too many high-risk labels. In Version 2, the raw risk score is still computed using the same weighted nonlinear formula, but a bias term is automatically calculated and added before applying the sigmoid function. This bias shifts the overall probability distribution upward or downward until the average predicted probability aligns with the desired 30% high-risk rate. In other words, the model's risk formula remains the same, but the threshold is adjusted so that the final dataset contains a more balanced mix of classes.

By adding this calibration step, Version 2 achieves a dataset distribution much closer to the intended **70%** low risk and **30%** high risk while preserving the nonlinear relationships between financial variables. This refinement directly addresses the imbalance issue identified in Version 1 without altering the neural network architecture itself. As a result, any improvements in the revised model performance can be interpreted as genuine gains in classification behavior rather than artifacts of class dominance.

7.3 Revised Model Results

After calibrating the dataset to achieve approximately a 70% low-risk and 30% high-risk distribution, the same MLP architecture and training configuration were applied. This ensures that performance differences between Version 1 and Version 2 are caused by improved dataset balance rather than changes in model structure.

7.3.1 Confusion Matrix Report

```
===== Test Results =====
Accuracy: 0.750

Confusion Matrix:
[[787  86]
 [227 150]]

Classification Report:
              precision    recall  f1-score   support

           0      0.776     0.901     0.834       873
           1      0.636     0.398     0.489       377

    accuracy                          0.750      1250
   macro avg      0.706     0.650     0.662      1250
weighted avg      0.734     0.750     0.730      1250
```

Figure 34: *Terminal results for V2*

When **train_mlp_v2.py** is executed, the program first prints a summary of test performance in the terminal (Figure 34) and then generates the updated training and evaluation plots. The terminal output reports an overall accuracy of **0.750**, followed by a confusion matrix and a classification report. Although accuracy is lower than in Version 1, this result is more meaningful because the dataset is now properly balanced. The metrics confirm that the model successfully trained, converged, and produced stable predictions rather than collapsing into a single-class strategy.

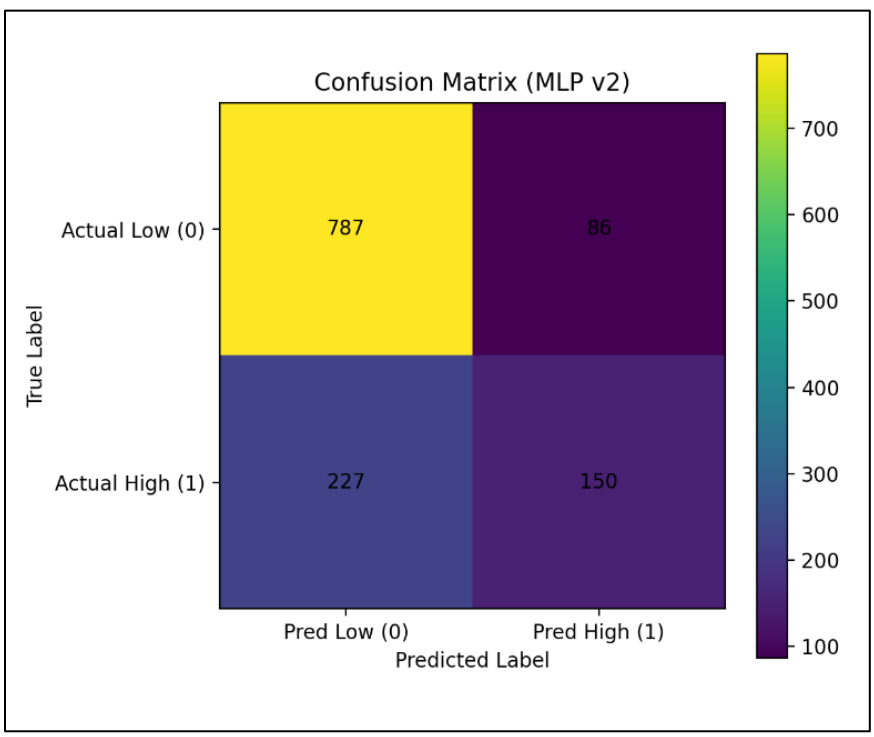

Figure 35: *Confusion Matrix for V2*

From the confusion matrix, each value represents:

- **787 (top-left)**
  - True Negatives (TN): actual low-risk (0) correctly predicted as low-risk (0)
- **86 (top-right)**
  - False Positives (FP): actual low-risk (0) incorrectly predicted as high-risk (1)
- **227 (bottom-left)**
  - False Negatives (FN): actual high-risk (1) incorrectly predicted as low-risk (0)
- **150 (bottom-right)**
  - True Positives (TP): actual high-risk (1) correctly predicted as high-risk (1)

Unlike Version 1, the model no longer predicts nearly all applicants as high risk (Figure 35). It correctly identifies **787** low-risk cases and **150** high-risk cases, demonstrating meaningful discrimination between classes. Although some high-risk applicants are still misclassified (**227 false negatives**), the model now produces balanced predictions instead of being heavily skewed toward one class. This already indicates that the dataset refinement successfully corrected the extreme bias observed in Version 1.

7.3.2 Training vs Validation Loss Curve

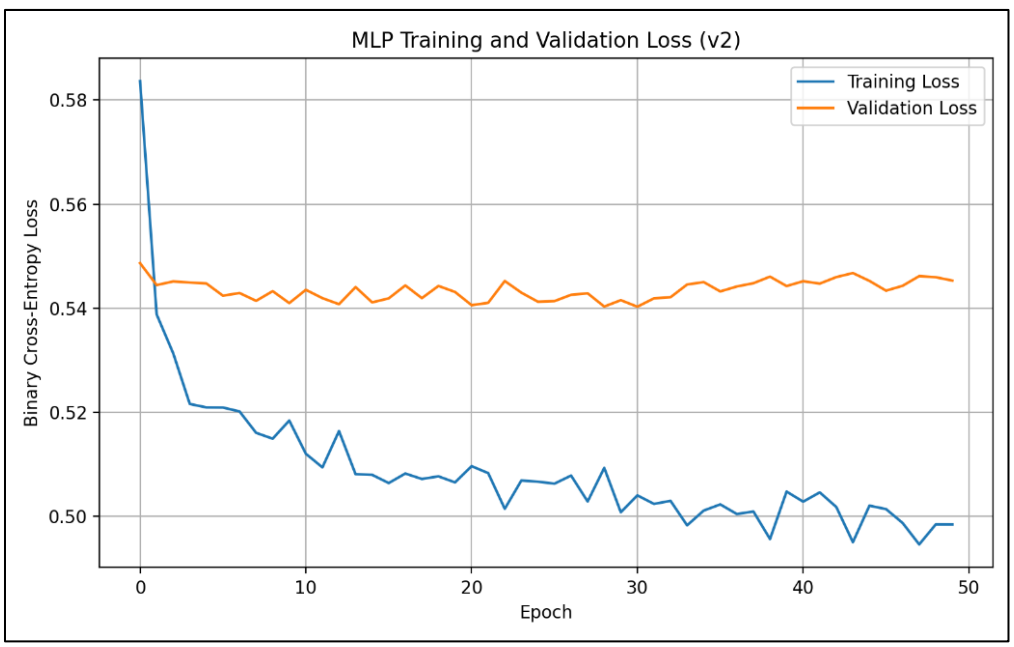

Figure 36: *MLP Training and Validation Loss Curve for V2*

The training behavior is visualized in the updated loss curve (Figure 36). Both curves represent binary cross-entropy loss. The training loss begins around **0.585** and drops sharply within the first few epochs to approximately **0.52**. This steep early decline is expected because the model starts with random weights and quickly learns the strongest signals in the dataset.

After the initial sharp decline during the first few epochs, the training loss gradually decreases toward approximately **0.498–0.505**. As epochs progress, the curve begins to flatten, indicating that the model is approaching convergence. Improvements become smaller with each epoch, which is expected once the network has captured the strongest patterns in the data.

The validation loss begins around **0.548**, decreases slightly in early epochs, and then stabilizes between **0.545–0.548**. Although it fluctuates mildly, it does not increase sharply or diverge from the training curve. The overall pattern shows a moderate but stable gap between training and validation loss. Because the validation loss does not rise dramatically, this suggests only very mild overfitting. More importantly, unlike Version 1, the model is no longer converging toward a trivial majority-class solution. Instead, it is learning a more balanced decision boundary.

The higher final loss compared to Version 1 (validation loss line in orange) reflects that the classification task is now genuinely challenging rather than artificially simplified by extreme class dominance. In other words, the model is solving a real discrimination problem rather than exploiting imbalance.

7.3.3 Predicted Probability Distribution by Class

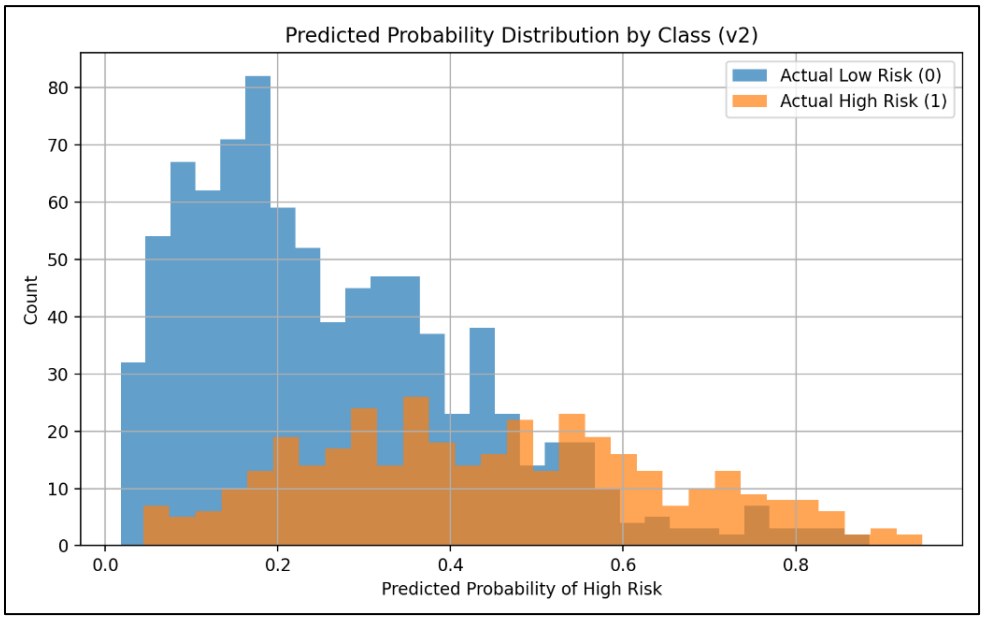

Figure 37: *Predicted Probability Distribution by Class (V2)*

The predicted probability distribution histogram provides deeper insight into how the model assigns confidence (Figure 37). For low-risk applicants (blue), most predicted probabilities fall between **0.05** and **0.35**, with a peak around **0.15–0.25**. This indicates that the model is generally confident when assigning low-risk labels. On the other hand, for high-risk applicants (orange), probabilities are more widely distributed, ranging approximately from **0.25 to 0.85**, with a concentration between **0.35–0.55**. The broader spread suggests greater uncertainty when identifying high-risk borrowers.

There is noticeable overlap between approximately 0.25 and 0.45. This overlap explains the presence of:

- False positives (low-risk predicted as high-risk)
- False negatives (high-risk predicted as low-risk)

Unlike Version 1, the distributions are no longer heavily skewed toward probability values near 1.0. Instead:

- Low-risk applicants cluster toward lower probabilities.
- High-risk applicants cluster toward higher probabilities.
- Separation between classes is imperfect but meaningful.

This indicates that the model is producing more calibrated and realistic probability estimates rather than extreme, biased predictions concentrated in one region.

7.3.4 Classification Report Analysis

The overall test accuracy is **0.750** (Figure 34), which is lower than the **0.906** (Figure 29) reported in Version 1. However, this reduction is expected and desirable. In Version 2, the model must meaningfully distinguish between two reasonably balanced classes instead of relying on a dominant class prediction. Therefore, accuracy now reflects true classification ability rather than imbalance bias.

For Class 0 (Low Risk):

- Precision: 0.776
- Recall: 0.901
- F1-score: 0.834
- Support: 873

The model correctly identifies approximately **90%** of low-risk applicants, indicating strong recall for this class. When it predicts low risk, it is correct about **77.6%** of the time. The F1-score of **0.834** demonstrates reliable and consistent performance for low-risk borrowers.

For Class 1 (High Risk):

- Precision: 0.636
- Recall: 0.398
- F1-score: 0.489
- Support: 377

For high-risk applicants, the model detects roughly **40%** of true high-risk cases. Precision of **0.636** indicates that when the model predicts high risk, it is correct about **64%** of the time. Although

recall remains moderate, this represents a substantial structural improvement over Version 1, where class discrimination was largely skewed. The performance is asymmetric but realistic. In credit risk modeling, it is often easier to identify clearly safe borrowers than borderline risky ones, which explains the higher recall for low-risk applicants.

8. Lessons Learned

8.1 The Importance of a Proper Dataset

One of the biggest lessons from this project is that model complexity does not guarantee meaningful performance. In Version 1, the Multi-Layer Perceptron was properly implemented, trained without crashing, and even achieved high numerical accuracy (0.906). On the surface, everything looked successful. But once the confusion matrix and classification report were examined, it became clear that the model had essentially learned to predict almost everything as high risk.

That moment was important. It showed that even when the code runs perfectly, the results can still be fundamentally flawed. This reinforces a core principle in machine learning: data quality and distribution often matter more than model architecture. A well-designed neural network cannot compensate for a poorly structured dataset. In Version 1, the imbalance created a shortcut solution for the model, allowing it to minimize loss without actually learning meaningful discrimination. Sometimes, the problem is the dataset, not the model.

8.2 Accuracy Can Be Misleading

Another major takeaway is that accuracy alone is not a reliable metric in classification tasks.

In Version 1:

- Accuracy = 0.906
- Recall for low-risk class = 0.009

That contrast is dramatic. A model that is correct 90% of the time but fails almost entirely on one class is not truly performing well. So, this project emphasized the importance of examining confusion matrices, precision and recall, F1-score, and also the probability distributions. Looking at multiple evaluation metrics provides a fuller picture of model behavior. In real-world applications such as credit risk assessment, relying only on accuracy could lead to serious decision-making errors that would cost millions or billions in the form of falsely approved loans. It was also honestly surprising how convincing that 0.906 looked before digging deeper.

8.3 Controlled Experimentation Matters

When moving to Version 2, the dataset size and random seed were intentionally kept the same. This ensured that improvements were due to the calibration adjustment rather than randomness. This was not just a coding decision but it was an experimental design decision.

If multiple variables are changed at once, it becomes impossible to identify the true cause of performance differences. By isolating the probability calibration step, the impact of the dataset refinement became clearly measurable. This project reinforced that machine learning is not just about writing models but it is about designing controlled experiments.

8.4 Probability Outputs Are More Informative Than Binary Labels

The probability distribution plots revealed another key insight: neural networks do not simply output class labels. They also output confidence levels.

In Version 1, probabilities were heavily skewed toward 1.0, indicating extreme confidence in high-risk predictions. In Version 2, the probabilities were more smoothly distributed and showed meaningful separation between classes.

This highlighted the importance of thinking beyond the 0.5 threshold. The decision boundary is just a rule layered on top of probabilities. Adjusting thresholds or applying cost-sensitive learning could significantly change model performance. It also made it clear that understanding probability behavior is just as important as understanding classification accuracy.

8.5 Nonlinearity Helps, But It Cannot Fix Bias

The MLP architecture included nonlinear activation functions, dropout layers, and adaptive optimization. These components allow the model to learn complex feature interactions, such as how a high debt-to-income ratio combined with a low credit score may increase risk more than either factor alone.

However, Version 1 demonstrated that nonlinearity alone does not guarantee fairness or balance. If the dataset is biased, the model will simply learn that bias more efficiently. For example, in Version 1, because the dataset contained a disproportionately large number of high-risk labels, the neural network learned that predicting “high risk” for almost every applicant minimized the loss. The model’s nonlinear layers and adaptive optimizer did not correct this issue. They actually reinforced it by converging quickly toward that dominant pattern.

Deep learning increases expressiveness, but it does not automatically increase validity. A powerful model can represent complex relationships, but if the underlying data distribution is flawed, it will faithfully learn and reproduce that flaw. That distinction is important.

9. Conclusion

This project demonstrated how a Multi-Layer Perceptron can model nonlinear relationships in structured credit risk data. While the initial version achieved high accuracy, deeper analysis revealed that the results were driven by class imbalance rather than meaningful classification. After refining the dataset to achieve a balanced class distribution, the model produced more realistic and interpretable performance metrics.

The key takeaway is that strong model architecture cannot compensate for flawed data design. Careful dataset construction, appropriate evaluation metrics, and controlled experimentation are essential for developing reliable machine learning systems. In the end, improving the data proved more impactful than increasing model complexity.

10. References

GeeksForGeeks. (2025, September 30). *Multi-Layer Perceptron Learning in Tensorflow*. Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/deep-learning/multi-layer-perceptron-learning-in-tensorflow/

GeeksForGeeks. (2025, October 4). *What is Adam Optimizer?* Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/deep-learning/adam-optimizer/